APRESENTAÇÃO DO PRODUTO · VERSÃO 1.1.1

QA Scout
Explore com missão. Volte com evidência.

Extensão do Chrome, gratuita e de código aberto, para teste exploratório baseado em sessões. Regista o que acontece enquanto explora e entrega o relatório e o bug prontos para a equipa.

0
pedidos de rede: os dados ficam no seu computador

4
idiomas: pt-BR, pt-PT, inglês e espanhol

176
testes automatizados no próprio projeto

Thamires Azevedo · QA Sénior · github.com/azevedothamirest/qa-scout

O CONCEITO

O que é teste exploratório?

É aprender sobre o sistema, desenhar os testes e executá-los **ao mesmo tempo**. Em vez de seguir um guião fechado, quem testa investiga com uma missão, observa o comportamento real e decide o passo seguinte pelo que acabou de ver.

Guião (script)

Verifica o que já se sabe que devia acontecer. Bom para regressão e automação.

Exploratório

Descobre o que ninguém pensou em escrever: combinações, bordas, fluxos reais. É onde aparecem os bugs mais caros.

Sessões com tempo marcado (SBTM)

O **Session-Based Test Management**, de Jon e James Bach, dá estrutura sem virar guião. Cada sessão tem três partes:

1. **Charter:** a missão, no modelo de Elisabeth Hendrickson.
2. **Sessão:** um bloco de tempo sem interrupção (30 a 90 min), anotando bugs, dúvidas, ideias, riscos e observações.
3. **Debriefing:** revisão das notas, divisão do tempo em Teste, Bug e Setup (TBS) e o que fica para a próxima sessão.

Explorar **o checkout com MB WAY** com **pagamento expirado, botão voltar e cupão** para descobrir **encomendas pagas que ficam pendentes**.

Divisão do tempo (TBS)



Teste 60% · Bug 25% · Setup 15%. Muito tempo em setup ou bug indica ambiente instável ou produto frágil: uma métrica que toda a equipa entende.

O PROBLEMA

Exploratório costuma ser trabalho invisível

Encontra os melhores bugs, mas deixa pouco rasto: o que foi explorado, durante quanto tempo, com que evidência? E reproduzir o bug depois vira arqueologia.

A SOLUÇÃO

O QA Scout anota enquanto explora

📸 Captura automática

Cliques, campos preenchidos e páginas abertas passam a notas com captura de ecrã, no painel lateral do Chrome. **O valor escrito nunca é lido.**

🧯 Erros de consola e rede

Erro de JavaScript, `console.error`, respostas 4××/5×× e falhas de ligação passam a notas sugeridas como bug ou risco.

🪜 Passos para reproduzir

Um clique monta a lista com as ações que vieram antes do bug.

🐞 Bug pronto para o Jira

"Copiar bug" gera título, ambiente, navegador, página, passos, obtido x esperado e os erros do mesmo minuto.

🃏 Baralho de heurísticas

18 cartas (tours de Whittaker, dados, interação, contexto) para quando a criatividade travar.

📄 Relatório da sessão

Debriefing com TBS e relatório em Markdown ou PDF para colar no Jira, Notion ou Slack.

Revê tudo. As notas automáticas chegam "por rever": confirme, edite ou elimine. Só as revistas entram no relatório.

PARA TODA A EQUIPA

Como cada área aplica

O exploratório não é exclusivo de QA. Com uma missão curta e 20 minutos, qualquer pessoa da equipa encontra o que o teste com guião não apanha.

QA

Sessões com missão e evidência

- Planeje a sprint em charters por risco: o que mais pode custar caro?
- Use o TBS no debriefing para mostrar onde o tempo foi parar.
- Abra bugs com passos e erros de consola já anexados, sem "não consigo reproduzir".
- Transforme o que a sessão descobriu em casos de regressão automatizada.

Dev

Explorar antes de abrir o PR

- Uma sessão de 15 min na própria feature: "explorar o formulário novo com dados inválidos para descobrir validações faltando".
- Bugs chegam com passos, ambiente, URL e o erro de JavaScript do mesmo minuto: menos ida e volta.
- Na revisão do PR, explore a branch em pré-produção e cole o relatório no PR.

Produto

Validar a experiência real

- Percorra a jornada como cliente antes da release: "explorar o onboarding com um utilizador novo para descobrir onde a pessoa encrava".
- Notas de **dúvida** e **ideia** viram perguntas de refinamento e backlog.
- O relatório mostra cobertura e riscos em linguagem de negócio, ótimo para decidir o go/no-go.

💡 Ritual sugerido: bug bash de 30 minutos

Cada pessoa (QA, dev, produto, design) escolhe um charter diferente, todos exploram ao mesmo tempo e a equipa junta os relatórios no fim. Rápido, barato e revela muito antes da release.

COMEÇAR

Instalação em 1 minuto

🧩 Extensão do Chrome (recomendado)

Instale pela Chrome Web Store, abra o painel lateral pelo ícone da bússola e comece uma sessão. O Chrome pede acesso aos sites na primeira utilização.

🖥️ Navegador de teste (sem mexer no seu Chrome)

Com Node.js 18+:

```
npx github:azevedothamirest/qa-scout https://seu-sistema
```

Passo a passo completo, código-fonte e tutorial: **github.com/azevedothamirest/qa-scout** (veja `docs/tutorial.md`).

COM IA

O Claude explora consigo (MCP)

O QA Scout tem um servidor **MCP** que deixa o Claude conduzir a sessão no navegador de teste: lê o ecrã, clica, preenche, consulta os erros capturados, anota o que encontra e devolve o relatório. A sessão fica no histórico para rever.

```
claude mcp add qa-scout -- npx -y github:azevedothamirest/qa-scout mcp
```

"Use o QA Scout para uma sessão de 20 minutos em https://homolog.minhaloja.pt: explorar o checkout, com cupões e MB WAY, para descobrir cobranças erradas. Mostre-me o relatório."

1. Lê

Entende o ecrã pela árvore de acessibilidade.

2. Explora

Clica e preenche como uma pessoa; a captura grava tudo.

3. Anota

Registra bugs, riscos e dúvidas e encerra com o relatório.

Corre no terminal (Claude Code ou Claude Desktop). O texto das páginas visitadas vai para o Claude: use só ambientes e dados de teste.

PODE USAR NO TRABALHO?

Privacidade e segurança

- ✓ Sem servidor, sem conta, sem telemetria: **nenhum pedido de rede** (bloqueada por CSP `connect-src 'none'`).
- ✓ Dados só no armazenamento local do navegador; saem apenas quando copia ou transfere algo.
- ✓ Valores escritos nunca são lidos; tokens e palavras-passe nos URLs são mascarados.
- ✓ Código aberto (licença MIT): é possível auditar tudo o que faz.
- ✓ Material para a equipa de segurança: `docs/seguranca.md`, termos de uso e `SECURITY.md`.

Resumo

QA

Evidência e rastreabilidade do exploratório.

Dev

Bugs reproduzíveis, com contexto técnico.

Produto

Visão real da experiência e dos riscos.

Explore com missão. Volte com evidência.

github.com/azevedothamirest/qa-scout · Thamires Azevedo · @qa.thami